# FICHA PRÁTICA LABORATORIAL 05 – AI-POWERED SOFTWARE ENGINEERING

**Objetivos**

- Executar a aplicação através de Docker e Docker Compose.
- Automatizar testes e builds com GitHub Actions.
- Criar e atualizar a documentação técnica.
- Preparar uma versão identificada do projeto.
- Validar o funcionamento da aplicação e diagnosticar problemas.
- Utilizar agentes de IA para apoiar estas atividades, mantendo a revisão e validação humana.

## PRÉ-REQUISITOS

- Projeto Matriz desenvolvido nas fichas práticas anteriores.
- Acesso ao repositório GitHub do projeto.
- Docker e Docker Compose instalados.
- Git e ambiente de desenvolvimento configurados.
- Agente de IA com acesso ao código do projeto.
- Testes automatizados desenvolvidos na Aula 04, quando aplicável.

Nota: todas as atividades devem respeitar a arquitetura e as tecnologias efetivamente utilizadas no Matriz. Os componentes apresentados nos exemplos da aula não devem ser introduzidos caso não sejam necessários ao projeto.

## EXERCÍCIO 01 — ANÁLISE E PREPARAÇÃO DO PROJETO

Antes de implementar os mecanismos de entrega, deverá ser realizada uma análise técnica ao estado atual do Matriz.

**Tarefas**

1. Identificar os principais componentes da aplicação.
2. Analisar as dependências, versões e comandos necessários à execução.
3. Identificar os serviços externos e as variáveis de ambiente.
4. Confirmar a existência de testes automatizados.
5. Definir uma estratégia para executar a aplicação num ambiente reproduzível.

**Prompt sugerido**

> Analisa integralmente o projeto Matriz e identifica a arquitetura, as tecnologias, os serviços, as dependências e os requisitos de execução. Propõe um plano para preparar a aplicação para entrega utilizando Docker, Docker Compose e GitHub Actions. Não alteres ficheiros antes de apresentares o plano para revisão.

**Resultado esperado:** plano técnico com os componentes identificados, os ficheiros a criar ou alterar e as etapas de implementação.

## EXERCÍCIO 02 — CONTAINERIZAÇÃO COM DOCKER

Implementar a containerização dos componentes aplicacionais do Matriz.

**Tarefas**

1. Criar ou atualizar os Dockerfiles necessários.
2. Configurar ficheiros .dockerignore.
3. Definir imagens base e instalar as dependências.
4. Configurar os comandos de arranque e as portas.
5. Construir as imagens Docker.
6. Executar os containers e validar o funcionamento.

**Comandos de referência**

- docker build — construir imagens.
- docker images — consultar imagens disponíveis.
- docker run — executar containers.
- docker ps — verificar o estado dos containers.
- docker logs — consultar logs de execução.

**Requisitos**

- Os containers devem iniciar corretamente.
- As dependências devem estar incluídas nas imagens.
- Não devem existir credenciais incluídas nos Dockerfiles.
- A aplicação deve manter o comportamento funcional anteriormente validado.

**Resultado esperado:** imagens Docker funcionais para os componentes aplicáveis do Matriz.

## EXERCÍCIO 03 — DOCKER COMPOSE E EXECUÇÃO DA APLICAÇÃO

Configurar a execução integrada da aplicação através de Docker Compose.

**Tarefas**

1. Criar o ficheiro compose.yaml.
2. Definir os serviços existentes na arquitetura.
3. Configurar as redes e a comunicação entre serviços.
4. Configurar volumes persistentes, quando necessários.
5. Criar um ficheiro .env.example com as variáveis de configuração.
6. Configurar healthchecks adequados aos serviços.
7. Executar e validar a aplicação completa.

**Comandos de validação**

```
docker compose config -q
docker compose build
docker compose up -d
docker compose ps
docker compose logs --tail=50
```

**Validações obrigatórias**

- A aplicação é acessível.
- Os componentes comunicam corretamente.
- As funcionalidades principais continuam operacionais.
- Os dados persistentes mantêm-se após a recriação dos containers, quando aplicável.
- Os logs não apresentam erros críticos por resolver.

**Resultado esperado:** aplicação Matriz executável através de Docker Compose.

## EXERCÍCIO 04 — INTEGRAÇÃO CONTÍNUA COM GITHUB ACTIONS

Criar uma pipeline para automatizar as verificações de qualidade e construção do projeto.

**Tarefas**

1. Criar o ficheiro .github/workflows/ci.yml.
2. Configurar a execução automática em push e pull_request.
3. Instalar as dependências necessárias.
4. Executar os testes automatizados existentes.
5. Validar o build dos componentes aplicacionais.
6. Construir as imagens Docker.
7. Verificar os resultados na secção Actions do GitHub.

**Requisitos**

- A pipeline deve falhar quando um teste obrigatório falha.
- A construção Docker só deve avançar após as verificações exigidas.
- O workflow deve utilizar permissões mínimas.
- Não devem ser expostos segredos ou credenciais.

**Validação adicional:** introduzir temporariamente uma falha controlada num teste, confirmar que a pipeline falha e, depois, corrigir o problema e voltar a executar as verificações.

**Resultado esperado:** pipeline CI funcional, com evidência de execução aprovada no GitHub.

## EXERCÍCIO 05 — DOCUMENTAÇÃO TÉCNICA E PREPARAÇÃO DA RELEASE

Preparar a documentação necessária para que outro programador consiga compreender e executar o Matriz.

**Tarefas**

1. Criar ou atualizar o ficheiro README.md.
2. Documentar a arquitetura em docs/architecture.md, incluindo um diagrama Mermaid.
3. Documentar a configuração e execução com Docker Compose.
4. Disponibilizar documentação da API com OpenAPI, caso exista uma API HTTP.
5. Criar ou atualizar o CHANGELOG.md.
6. Preparar notas de lançamento para uma versão candidata do Matriz.

**O README deverá incluir**

- Descrição e funcionalidades da aplicação.
- Tecnologias utilizadas.
- Pré-requisitos.
- Instruções de instalação e configuração.
- Comandos de execução.
- Instruções para executar os testes.
- Referências para a documentação adicional.

**Prompt sugerido**

> Analisa a implementação atual do Matriz e cria ou atualiza a documentação técnica. Garante que todas as funcionalidades, tecnologias, comandos e configurações documentadas correspondem ao código existente. Identifica informações não confirmadas e apresenta-as para revisão. Não inventes funcionalidades nem resultados de testes.

**Resultado esperado:** documentação atualizada e proposta de versão identificada através de Semantic Versioning. Não é necessário publicar uma release de produção.

## EXERCÍCIO 06 — VALIDAÇÃO FINAL E TROUBLESHOOTING

Realizar uma verificação global à preparação do Matriz para entrega.

**Tarefas**

1. Executar a aplicação a partir das instruções do README.
2. Confirmar o estado dos containers.
3. Testar as funcionalidades essenciais.
4. Confirmar os resultados da pipeline CI/CD.
5. Consultar os logs e identificar possíveis avisos ou erros.
6. Verificar a ausência de credenciais nos ficheiros versionados.
7. Registar os problemas encontrados e as respetivas correções.

Solicitar ao agente de IA uma revisão final do projeto, incluindo documentação, configuração Docker, pipeline e resultados dos testes.

**Resultado esperado:** relatório sucinto de validação, indicando os requisitos cumpridos, os problemas identificados e eventuais limitações por resolver.

## DESAFIOS ADICIONAIS (OPCIONAIS)

Para os formandos que concluírem os exercícios obrigatórios:

- **Desafio A — Segurança:** integrar uma ferramenta de análise de dependências ou imagens Docker na pipeline.
- **Desafio B — Observabilidade:** implementar logs estruturados e verificações adicionais de disponibilidade.
- **Desafio C — Releases:** configurar a publicação automática de uma imagem Docker no GitHub Container Registry, utilizando uma release de teste e permissões adequadas.

## ENTREGA

A entrega deverá ser efetuada através do repositório GitHub do projeto Matriz.

O repositório deverá conter:

- Código-fonte atualizado.
- Dockerfiles e ficheiros .dockerignore.
- Configuração Docker Compose.
- Ficheiro .env.example, sem credenciais reais.
- Workflow GitHub Actions.
- README e documentação técnica.
- Changelog e notas da versão candidata.
- Evidências dos testes e verificações realizadas.

Deverá ainda ser apresentado um breve relatório com:

1. Alterações implementadas.
2. Resultados dos testes e da pipeline.
3. Problemas encontrados e soluções aplicadas.
4. Utilização dos agentes de IA.
5. Limitações ou funcionalidades pendentes.

Não devem ser incluídos na entrega: passwords, tokens, ficheiros .env com valores sensíveis ou outros dados confidenciais.

## CONSIDERAÇÕES FINAIS

A utilização de Inteligência Artificial é incentivada em todas as atividades.

No entanto, os formandos deverão compreender as configurações produzidas, justificar as decisões técnicas e validar os resultados obtidos.

**O objetivo não é apenas conseguir executar o Matriz, mas demonstrar que a aplicação pode ser construída, testada, documentada e preparada para entrega através de um processo de engenharia reproduzível e controlado.**

**Bom trabalho! ☺**